# The Treacherous Transpiler: Compiler Supply-Chain Integrity & Translation Validation

—

# BreaQ 2026 — Challenge 2 Technical Investigation & Defense Report

**Team:** $\mu - Qbit$
Participants: Richard Watry, Dylane Mahazoasy
Date: October 25, 2026

## Abstract

This report presents our technical investigation, fault localization, minimal exploit reproduction, and translation-validation defense for BreaQ 2026 Challenge 2 (*The Treacherous Transpiler*). Using local simulation in Python 3.13.7 and Qiskit 2.6.1, we isolate a supply-chain integrity violation inside the third-party compiler pass `03:vendor-echo-balance` (`_EchoBalance`) and demonstrate how full standalone-unitary translation validation (`verify_compilation` and `guarded_compile`) prevents silent semantic corruption.

## 1 Executive Summary, Threat Model & Service Contract

### 1.1 Security Claim

Based on our experimental investigation, fault localization, and defense evaluation (all executed via local simulation in Python 3.13.7 / Qiskit 2.6.1), our core security finding is:

> **Security Claim:** *The suspect compiler's `03:vendor-echo-balance` pass (`_EchoBalance`) violates standalone unitary equivalence by negating the rotation angle ($\theta \rightarrow -\theta$) of the first $R_z(\theta)$ gate on the target wire of a consecutive CX–$R_z$–CX pattern whenever $\theta$ lies in the seed-dependent window $[0.325, 0.825]$ rad; because this relative phase flip can evade $Z$-basis histograms (`SCAN-D`, `SCAN-F`) and all-zero state fidelity (`SCAN-F`), compilation must be guarded by an independent full-unitary translation validator (`verify_compilation`/`guarded_compile`).*

### 1.2 Threat Model and Trust Boundary

In this architecture, a trusted application submits a bound, measurement-free quantum circuit ($1 \ldots 6$ qubits) across a trust boundary to an untrusted remote compiler service containing a third-party vendor plugin. The candidate circuit returned by the compiler is then verified locally before execution on a trusted local simulator. Table 1 summarizes the threat model recorded in `submission.json`.

### 1.3 Warm-Up Questions: Legitimate Rewriting vs. Required Security Property

1. **Why can two different-looking circuits be semantically equivalent, and what should a verifier accept?**

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

| Element | Specification |
|---|---|
| **Protected Asset** | The semantic integrity of the submitted quantum computation (the standalone $2^n \times 2^n$ unitary operator $U$ acting on logical wires $q_0 \dots q_{n-1}$). |
| **Attacker Capability** | Control over a third-party compiler plugin (`03:vendor-echo-balance`) able to inspect the circuit DAG and selectively alter gates or rotation angles while producing valid native-basis syntax. |
| **Trust Boundary** | Between the trusted local environment (source circuit, local Qiskit math routines, independent verifier, simulator) and the untrusted compiler service (`service.compile`). |
| **Security Property** | Exact standalone-unitary equivalence up to global phase ($U_{\text{candidate}} = e^{i\phi} U_{\text{source}}$ within numerical tolerance) and strict compliance with the $1 \dots 6$ qubit contract. |
| **Trusted Components** | Source circuit, local Qiskit `quantum_info` routines (`Operator`, `Statevector`, `state_fidelity`), local simulator, operating system, and notebook verifier. |
| **Out of Scope** | Classical bits, dynamic control, ancillas, physical placement, SWAP routing, coherent control of the entire compiled circuit, and utility-scale ($n > 6$) verification. |

Table 1: Threat model worksheet for Challenge 2 (`submission.json`).

Transpilers operate on a Directed Acyclic Graph (DAG) to lower high-level gates into hardware-native basis gates $\{\texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ and cancel redundant operations. In the warm-up experiment, a single Hadamard gate $H$ on $q_0$ is rewritten via Euler decomposition as $R_z(\pi/2)\sqrt{X}\,R_z(\pi/2)$ with an added global phase of $+\pi/9$. Although their gate counts (1 vs. 3), syntax, and SHA-256 QPY digests (`circuit_digest`) differ, their unitary matrices satisfy $U_{\text{rewritten}} = e^{i\phi} U_{\text{original}}$, so `Operator(original).equiv(Operator(rewritten))` returns `True`. A sound verifier must accept any in-contract circuit whose unitary operator matches the source up to a global phase factor $e^{i\phi}$.

2. **Which claim is stronger: "same observed output distribution" or "same standalone unitary"?**

   **"Same standalone unitary"** is strictly stronger and is required by the service contract. Measuring a circuit initialized in $|0 \dots 0\rangle$ in the computational ($Z$) basis only samples the probability distribution $P(x) = |\langle x|U|0 \dots 0\rangle|^2 = |U_{x,0}|^2$. This checks only the *magnitudes* of the first column of $U$, leaving all relative phases on $|0 \dots 0\rangle$ and all entries of the remaining $2^n - 1$ columns of $U$ completely untested.

## 2 Level 1 — Incident Triage & Core Investigation

### 2.1 Triage Observations vs. Security Conclusions

We first compared 512-shot local simulator observations (`sample_counts`) between source and suspect compiled circuits:

- `CAL-BELL`**:** Source and candidate yield identical counts `{'00': 261, '11': 251}`.
- `SCAN-D`**:** Source and candidate yield identical counts `{'100': 123, '110': 129, '101': 138, '111': 122}`.
- `SCAN-B`**:** Source counts `{'100': 196, '111': 190, '101': 65, '110': 61}` are inverted in the candidate to `{'101': 206, '110': 194, '111': 57, '100': 55}`.

**Observation vs. Conclusion:** A finite-shot histogram is a single-input, single-basis observation. While the dramatic inversion on `SCAN-B` strongly indicates a deterministic semantic alteration triggered by a circuit feature in `SCAN-B`, the matching histograms on `CAL-BELL` and `SCAN-D` *cannot* establish that those compilations are uncompromised.

### 2.2 Core Workload Comparison Table

To evaluate all eight workloads systematically without relying on workload names, we implemented `inspect_compilation(source, candidate)` to compute gate counts, circuit depth, exact $Z$-basis Total Variation Distance (TVD), all-zero input state fidelity $F(|\psi_{\text{src}}\rangle, |\psi_{\text{cand}}\rangle) = |\langle 0^{\otimes n}|U_{\text{src}}^{\dagger}U_{\text{cand}}|0^{\otimes n}\rangle|^2$, and full unitary equivalence (`Operator.equiv`). Table 2 reports the complete results exported to `submission_artifacts/comparison.csv`.

| Case ID | Src Gates | Cand Gates | Src Depth | Cand Depth | Exact TVD ($Z$) | $\|0^{\otimes n}\rangle$ Fidelity | Unitary Equiv |
|---|---|---|---|---|---|---|---|
| CAL-BELL | 2 | 4 | 2 | 4 | $1.67 \times 10^{-16}$ | 1.000000 | True |
| LEGACY-REDUNDANCY | 6 | 0 | 4 | 0 | $2.22 \times 10^{-16}$ | 1.000000 | True |
| SCAN-A | 8 | 12 | 6 | 8 | $2.22 \times 10^{-16}$ | 1.000000 | True |
| SCAN-B | 8 | 12 | 6 | 8 | **0.543835** | **0.704244** | False |
| SCAN-C | 8 | 12 | 6 | 8 | $4.72 \times 10^{-16}$ | 1.000000 | True |
| SCAN-D | 6 | 10 | 4 | 6 | $2.50 \times 10^{-16}$ | **0.704244** | False |
| SCAN-E | 8 | 12 | 6 | 8 | $2.57 \times 10^{-16}$ | 1.000000 | True |
| SCAN-F | 4 | 4 | 3 | 3 | 0.000000 | 1.000000 | False |

Table 2: Multi-metric comparison across all 8 workloads (local simulation, `comparison.csv`).

### 2.3 Why Metrics Disagree on SCAN-D and SCAN-F

Table 2 reveals three compromised workloads (SCAN-B, SCAN-D, SCAN-F) that illustrate a strict hierarchy of verification metrics:

1. SCAN-B **(Caught by all metrics):** Applies $H(0), H(1)$, then the $\text{CX}(0,1)$–$R_z(0.575, 1)$–$\text{CX}(0,1)$ gadget, followed by the $Y$-basis analyzer $S^{\dagger}(1)H(1)$. Negating $\theta \to -\theta$ converts relative phases into computational-basis amplitude differences, producing TVD $= \sin(0.575) = 0.543835$ and state fidelity $\cos^2(0.575) = 0.704244$.
2. SCAN-D **(Disagreement #1 — $Z$-basis histogram fails, State Fidelity succeeds):** Identical to SCAN-B but omits the final $S^{\dagger}(1)H(1)$ analyzer. On input $|000\rangle$, the state after the gadget is $\frac{1}{2}e^{-i\theta/2}(|100\rangle + |111\rangle) + \frac{1}{2}e^{+i\theta/2}(|101\rangle + |110\rangle)$. Flipping $\theta \to -\theta$ conjugates the relative phases between the even- and odd-parity branches without changing the $Z$-basis probabilities (all four states remain at probability $1/4$, so TVD $= 0$). However, the statevector overlap drops to $|\langle\psi_{\text{src}}|\psi_{\text{cand}}\rangle|^2 = \cos^2(\theta) = 0.704244$.
3. SCAN-F **(Disagreement #2 — Both $Z$-basis histogram AND $|000\rangle$ State Fidelity fail):** Omits both the initial $H(0), H(1)$ superposition gates and the final analyzer. Because $|000\rangle$ is already a computational-basis eigenstate of $R_{ZZ}(\pm\theta) = \text{CX}(0,1)R_z(\pm\theta, 1)\text{CX}(0,1)$, the source circuit outputs $e^{-i\theta/2}|100\rangle$ while the compromised circuit outputs $e^{+i\theta/2}|100\rangle$. On this single input vector, the phase difference $e^{i\theta}$ acts as an unobservable overall phase ($|\langle\psi_{\text{src}}|\psi_{\text{cand}}\rangle|^2 = |e^{i\theta}|^2 = 1.0$, TVD $= 0.0$). Only `Operator.equiv` (`False`) detects that the $8 \times 8$ unitary matrices assign opposite phases to $\{|00\rangle, |11\rangle\}$ versus $\{|01\rangle, |10\rangle\}$.

## 3 Level 2 — Fault Localization & Trigger Discovery

### 3.1 Locating the First Contract Violation

Using detached trace snapshots from `service.trace(workloads["SCAN-B"])`, we evaluated consecutive stage transitions via `Operator.equiv` (Table 3, `stage_localization.csv`).

**Dependence on Trace Instrumentation:** Attributing the fault specifically to stage `03:vendor-echo-balance` depends on trusting that `service.trace` returns authentic intermediate snapshots. In an adversarial remote deployment, a compromised service could return fake intermediate traces; however, *detecting* the violation requires only comparing the trusted source (`00:input`) against the final candidate (`04:basis-lowering`), which is completely independent of trace instrumentation.

| Step | Pipeline Transition | Unitary Equiv | Observed Transformation on SCAN-B |
|---|---|---|---|
| 1 | 00:input -> 01:normalization | True | Removes barriers (RemoveBarriers); semantics preserved. |
| 2 | 01:normalization -> 02:phase-convention | True | Adds benign global phase $+\pi/7$; allowed by contract. |
| 3 | 02:phase-convention -> 03:vendor-echo-balance | False | **First violation:** mutates $R_z(0.575)$ on $q_1$ to $R_z(-0.575)$. |
| 4 | 03:vendor-echo-balance -> 04:basis-lowering | True | Faithfully lowers the already-corrupted circuit to $\{\texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$. |

Table 3: Consecutive stage equivalence checks on SCAN-B (stage_localization.csv).

### 3.2 Trigger Discovery & Counterfactual Verification

Inspecting _EchoBalance in breaq_ch2/service.py and testing one feature at a time established the exact activation rule:

- **Wire-Local Sequence:** On a qubit wire $w$, three consecutive operations in dag.nodes_on_wire(w, only_ops=True) (after stage 01:normalization removes barriers) must have operation names ('cx', 'rz', 'cx').
- **Target-Qubit Role:** Both CX gates must share identical (control, target) qubits (tuple(left.qargs) == tuple(right.qargs)), and the $R_z$ gate must reside on the CX **target** wire $w$ (left.qargs[1] == wire).
- **Parameter Window:** The rotation angle $\theta$ must satisfy lower $\leq \theta \leq$ upper, where:

$$\text{lower} = 0.25 + 0.025 \times (\text{seed} \bmod 7), \qquad \text{upper} = \text{lower} + 0.50$$

  For DEMO-TEAM (seed = 2026, $2026 \bmod 7 = 3$), the activation window is $[0.325, 0.825]$ rad (midpoint $\theta = 0.575$ rad). This explains why SCAN-A ($\theta = 0.1625 < 0.325$), SCAN-E ($\theta = 1.075 > 0.825$), and SCAN-C ($R_z$ on control wire $q_0$) are not triggered.

Table 4 summarizes our six controlled counterfactual trigger experiments exported to submission_artifacts/trigger_experiments.csv.

| Controlled Counterfactual Experiment (trigger_experiments.csv) | Predicted | Triggered |
|---|---|---|
| 1. **MATCH:** Echo CX(0,1) - Rz(0.575, 1) - CX(0,1) on TARGET qubit $q_1$ with valid angle | True | True |
| 2. **MISS:** Angle falls outside the activation window ($\theta$ = upper + 0.1 = 0.925 rad) | False | False |
| 3. **MISS:** Wrong rotation axis (Rx(0.575, 1) instead of Rz(0.575, 1)) | False | False |
| 4. **MISS:** Rz(0.575, 0) placed on the CONTROL wire $q_0$ instead of TARGET $q_1$ | False | False |
| 5. **MISS:** Sequence interrupted by another gate (H(1)) on target wire $q_1$ | False | False |
| 6. **MATCH:** Spectator gate X(2) on unrelated wire $q_2$ preserves DAG adjacency on $q_1$ | True | True |

Table 4: Controlled counterfactual experiments verifying the trigger hypothesis.

## 4 Minimal Reproducer & Observable Witness

### 4.1 Artifact 1: Minimal Activating Circuit (minimal_source.qpy)

Our minimization criterion was to minimize both qubit width ($n$) and gate count ($g$) while still triggering a standalone unitary violation (Operator.equiv == False). Starting from the 3-qubit, 8-gate SCAN-B circuit, we removed the spectator wire $q_2$ and gate $X(2)$, the initial $H(0), H(1)$ preparation, and the final $S^\dagger(1), H(1)$ analyzer, arriving at the 2-qubit, 3-gate circuit (minimal_source.qpy):

$$\texttt{minimal_source}: \quad \text{CX}(0,1) \longrightarrow R_z(0.575, 1) \longrightarrow \text{CX}(0,1)$$

**Minimality Proof by Deletion:** Removing any single gate from `minimal_source` restores exact equivalence (`remove_left_cx_equiv = True`, `remove_right_cx_equiv = True`, `remove_middle_rz_equiv = True`), proving that 2 qubits and 3 gates is strictly minimal. On the default $|00\rangle$ input, the bare subroutine exhibits `zero_input_tv = 0.0` and `zero_input_state_fidelity = 1.0` despite `unitary_equivalent_to_source = False`.

### 4.2 Artifact 2: Trusted External Preparation and Analysis Witness

To make the silent corruption of `minimal_source` macroscopically observable on a standard $Z$-basis simulator without modifying the submitted subroutine itself, `attach_trusted_witness(core_circuit)` wraps the compiled circuit *externally* in the trusted local harness:

1. **Trusted Preparation (pre-circuit):** Apply $H(1)$ to $|00\rangle$, preparing $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$ (in Qiskit's $|q_1 q_0\rangle$ bit ordering).
2. **Untrusted Compiled Subroutine:** Execute the compiled candidate for CX$(0,1)$–$R_z(\pm\theta, 1)$–CX$(0,1)$, which maps the state to $\frac{1}{\sqrt{2}}(e^{\mp i\theta/2}|00\rangle + e^{\pm i\theta/2}|10\rangle)$.
3. **Trusted Analysis (post-circuit):** Apply $S^\dagger(1)$ followed by $H(1)$ prior to $Z$-basis measurement, converting the relative phase $\pm\theta$ into computational-basis probabilities:

$$P_{\text{source}}(\texttt{00}) = \frac{1+\sin\theta}{2} \approx 0.7719, \qquad P_{\text{source}}(\texttt{10}) = \frac{1-\sin\theta}{2} \approx 0.2281$$

For the suspect compiled circuit ($\theta \to -\theta$), the distribution flips to $P_{\text{suspect}}(\texttt{00}) \approx 0.2281$ and $P_{\text{suspect}}(\texttt{10}) \approx 0.7719$, producing an exact Total Variation Distance of:

$$\text{TVD}_{\text{witness}} = |\sin(0.575)| = 0.543835, \qquad F_{\text{witness}} = \cos^2(0.575) = 0.704244$$

In 512-shot local sampling, the source witness yields `{'00': 400, '10': 112}` whereas the suspect witness yields `{'00': 113, '10': 399}`.

## 5 Level 3 — Verification Gate & Defense Evaluation

### 5.1 Design of `verify_compilation` and `guarded_compile`

We implemented a standalone, fail-closed translation validator in both the notebook and `submission_artifacts/student_api.py`:

- **Contract Validation (`_is_valid_contract_circuit`):** Verifies that both `source` and `candidate` are `QuantumCircuit` instances with $1 \leq$ num_qubits $\leq 6$, `num_clbits == 0`, no unbound `parameters`, `layout is None`, $\leq 500$ instructions, a finite real `global_phase`, only allowed contract gates, and finite real gate parameters (explicitly rejecting booleans, `NaN`, and $\pm\infty$).
- **Semantic Equivalence Check:** Verifies matching circuit widths (`source.num_qubits == candidate.num_qubits`) and checks `Operator(src_clean).equiv(Operator(cand_clean))`, which validates $U_{\text{cand}} = e^{i\phi} U_{\text{src}}$ across the entire $2^n$-dimensional Hilbert space. Any exception or check failure returns `False`.
- **Execution Wrapper (`guarded_compile`):** Compiles the circuit, runs `verify_compilation(source, candidate)`, and raises `ValueError("Compilation refused: integrity not established")` if verification fails.

### 5.2 Confusion Matrix and Test Suite Results

We evaluated `verify_compilation` across 24 test cases spanning both negative (legitimate/equivalent) and positive (compromised/invalid) classes (Table **??**):

- **Negative Class (TN $= 15$, FP $= 0$):** All 8 baseline compilations (`baseline_*`), all 5 uncompromised suspect compilations (`CAL-BELL`, `LEGACY-REDUNDANCY`, `SCAN-A`, `SCAN-C`,

SCAN-E), a manual Euler basis rewrite of $H$ with $+\pi/5$ global phase, and a circuit pair differing by barrier removal and a $-0.73$ rad global phase shift.

- **Positive Class (TP $= 9$, FN $= 0$):** All 3 compromised suspect workloads (SCAN-B, SCAN-D, SCAN-F), the 2-qubit minimal reproducer, a synthetic relative-phase mutation ($HSH$ vs. $HS^\dagger H$), a width mismatch (2q vs. 3q), an out-of-contract 7-qubit circuit, a circuit containing classical bits/measurements, and a non-circuit string object.

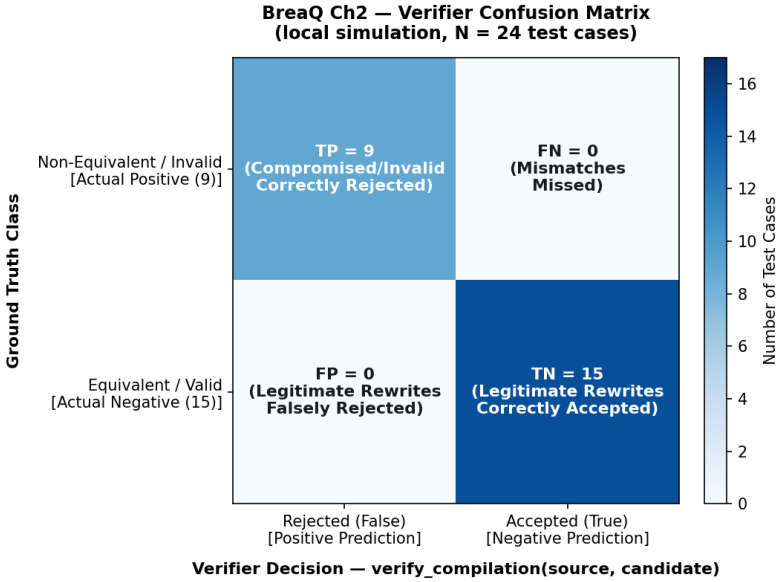



Figure 1: Confusion matrix for `verify_compilation` across the 24-case evaluation suite (`submission_artifacts/confusion_matrix.png`).

### 5.3 Recorded Limitations of the Verifier

1. **Exponential Scaling:** Constructing dense $2^n \times 2^n$ matrices via `Operator` requires $\mathcal{O}(4^n)$ memory and $\mathcal{O}(8^n)$ matrix-multiplication time, which is fast for $n \leq 6$ ($64 \times 64$) but intractable for utility-scale quantum processors.
2. **Coherent-Control Global-Phase Blindness:** `Operator.equiv` ignores global phase $e^{i\phi}$, which is valid for standalone execution but unsafe if the compiled circuit is later embedded as a coherently controlled subroutine (`controlled-U`), where global phase becomes a relative phase.
3. **Fixed Logical Wire Mapping:** The verifier assumes a 1-to-1 identity mapping between input and output logical wires and would reject legitimate hardware compilations that permute physical qubits via SWAP routing or allocate ancillas unless layout permutations are tracked.
4. **Numerical Tolerance Boundary:** Floating-point tolerances (`atol`/`rtol`) in `Operator.equiv` mean an adversary could theoretically chain sub-tolerance angle perturbations that individually pass numerical comparison.

## 6 Advanced Extensions

### 6.1 Extension 1: Trigger Boundary Angle Sweep

We swept $\theta \in [0.10, 1.05]$ rad across 96 points at a scan resolution of $\Delta\theta = 0.01$ rad (`trigger_boundary_sweep.csv`). As shown in Figure 2, the suspect compiler transitions sharply into a non-equivalent unitary for $\theta \in [0.33, 0.82] \pm 0.01$ rad, consistent with the exact seed-2026 interval $[0.325, 0.825]$ rad.

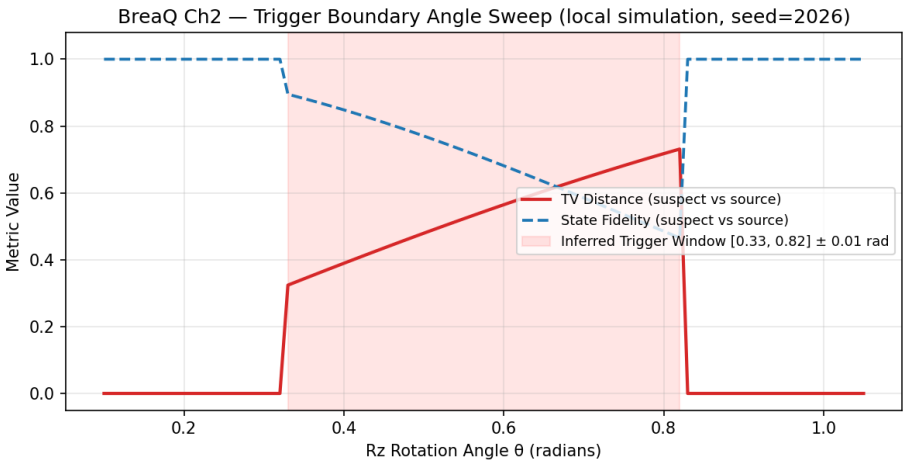



Figure 2: Trigger boundary angle sweep at resolution $\Delta\theta = 0.01$ rad (local simulation, `seed=2026`).

### 6.2 Extension 2: Semantic Metamorphism

We tested three logically equivalent source syntaxes implementing the exact same $R_{ZZ}(\theta)$ unitary up to global phase:

- `canonical_cx_rz_cx` ($\mathrm{CX}(0,1)$–$R_z(\theta,1)$–$\mathrm{CX}(0,1)$): **Triggers malware** (`equiv = False`).
- `phase_gate_cx_p_cx` ($\mathrm{CX}(0,1)$–$P(\theta,1)$–$\mathrm{CX}(0,1)$): **Evades malware** (`equiv = True`).
- `split_rotation_cx_rz_rz_cx` ($\mathrm{CX}(0,1)$–$R_z(\theta/2,1)$–$R_z(\theta/2,1)$–$\mathrm{CX}(0,1)$): **Evades malware** (`equiv = True`).

**Why this is diagnostic evidence, not a defense:** Syntax sensitivity proves the violation is caused by a pattern-matching compiler pass (`op.name == 'rz'`) rather than simulator noise, but rewriting source gates is not a robust defense because earlier normalization passes in a real compiler can canonicalize `p(theta)` or merge adjacent $R_z(\theta/2)$ gates back into a single `rz(theta)` before the malicious plugin executes.

### 6.3 Extension 3: DAG-Aware Pattern & Gate-Count Counterexamples

- **Wire Adjacency vs. Instruction-List Adjacency:** Inserting `barrier()` gates between `cx` and `rz` still activates the trojan because stage `01:normalization` (`RemoveBarriers`) strips them before stage `03`. Similarly, inserting $X(0)$ on the control wire $q_0$ between the gates separates the instructions in the printed circuit list but leaves `('cx', 'rz', 'cx')` adjacent on target wire $q_1$ in `dag.nodes_on_wire(q1)`, so the trojan **still activates**. Conversely, inserting `id(1)` on target wire $q_1$ breaks DAG adjacency on $q_1$ and prevents activation.
- **Gate-Count Counterexample:** `LEGACY-REDUNDANCY` reduces gate count from $6 \to 0$ and depth from $4 \to 0$ while remaining $100\%$ unitary equivalent, whereas `SCAN-F` leaves gate count (4) and depth (3) unchanged while silently corrupting the unitary. Gate count or depth reduction is never an integrity certificate.
- **Scalability Proposals:** Beyond $n = 6$ qubits, one can use (a) *randomized Pauli/Clifford probing* (polynomial for near-Clifford circuits, but only a probabilistic witness that can miss sparse-subspace trojans like `SCAN-F` on $|000\rangle$), or (b) *local DAG-window translation validation* ($\mathcal{O}(m \cdot 4^k)$ across $m$ local $k$-qubit rewrites, which scales linearly in circuit size but requires authenticated stage transitions).

## 7 Claim Sheet & Oral-Defense Summary

Table 5 synthesizes our five primary claims, their experimental evidence, trusted assumptions, and limitations (`submission.json`).

| **Claim** | Experimental Evidence | Trusted Assumption | Limitation |
|---|---|---|---|
| **1. Single-basis histograms and $\|0^{\otimes n}\rangle$ fidelity are insufficient.** | `SCAN-D` has TVD $= 0$ but Fidelity $= 0.704$; `SCAN-F` has TVD $= 0$ and Fidelity $= 1.0$, yet `Operator.equiv = False`. | Contract requires full standalone unitary equivalence; local `Operator` math is trusted. | Dense `Operator` matrices scale as $\mathcal{O}(4^n)$ space and $\mathcal{O}(8^n)$ time ($n \leq 6$). |
| **2. First violation occurs at `02:phase-convention -> 03:vendor-echo-balance`.** | Adjacent `Operator.equiv` is `True` for stages $00 \to 01$, $01 \to 02$, and $03 \to 04$, and `False` only at $02 \to 03$. | `service.trace` snapshots reflect true intermediate states for stage attribution. | A remote compiler could forge diagnostic trace logs while returning a bad final circuit. |
| **3. Trojan triggers on DAG target-wire `('cx','rz','cx')` with $\theta \in [0.325, 0.825]$.** | 6 counterfactuals (`trigger_experiments.csv`) and 96-point sweep ($[0.33, 0.82] \pm 0.01$ rad). | Compiler pass is deterministic for a fixed instance seed. | Identifying one trigger does not prove absence of other dormant triggers. |
| **4. 2-qubit, 3-gate `CX-Rz-CX` is minimal; external witness reveals it.** | Deleting any of the 3 gates restores equivalence; external $H(1) + S^\dagger(1)H(1)$ raises TVD from 0.0 to 0.5438. | Prep and analysis gates are applied in the trusted local environment after compilation. | A single witness only tests one subspace and does not replace full verification. |
| **5. `verify_compilation` & `guarded_compile` block all tested violations.** | 24-case suite achieves TP $= 9$, FN $= 0$, TN $= 15$, FP $= 0$ (`student_api.py`). | Logical wires have a fixed 1-to-1 identity mapping without routing or ancillas. | Ignores global phase (unsafe under coherent control) and does not scale to large $n$. |

Table 5: Structured claim sheet from `02_treacherous_transpiler.ipynb` and `submission.json`.

**Oral-Defense Quick Reference**

- **Why does a relative sign change ($\theta \to -\theta$) not change every experiment?** Because on computational-basis eigenstates (such as $|000\rangle$ in `SCAN-F`) or without an interfering measurement basis (`SCAN-D`), a diagonal phase rotation $R_{ZZ}(\pm\theta)$ either imparts an unobservable overall phase on a single basis vector or alters only relative phases that disappear under $Z$-basis projective measurement $|\psi_x|^2$.
- **What changes if the adversary also controls the simulator or verification environment?** Post-compilation verification (`verify_compilation`) relies on the integrity of the local Qiskit `Operator` engine and simulator. If the adversary controls the verifier or simulator, they can spoof `Operator.equiv` to return `True` or invert the measurement basis during simulation, requiring cross-machine verification or cryptographic verifiable quantum computation.